WINMOL stem segmentation — method corrections and scale-augmentation results

Christian Winkelmann

14 August 2026

Contents

1 Executive summary

Two independent bodies of work, in the order they should be read.

1.1 Part I — Corrections to the published WINMOL method

Read directly from `WINMOL_segmentor` (R) and `WINMOL_Analyzer` (Python), with every value quoted from source or measured by running it. Five items change how the published results should be read, and three of them are places where the papers and the code disagree.

#	Finding	Consequence
1	**The composite F1/BCE loss does not train.** `F1Score_loss` calls the *metric*, which applies k_round; a rounded value has zero derivative almost everywhere.	The model trains on **plain BCE**. The term appears in the reported loss and is absent from the optimisation. Verified three ways, including an 8-epoch R run agreeing with pure BCE to **eight decimal places**.
2	**The split is a random 80/20 over tiles**, drawn per stage — not site-level, not spatial. With 100× oversampling, tiles overlap.	Early stopping, the LR schedule and checkpoint selection all read an optimistic validation signal.
3	**Tile side length is 15 m, not ≈15.4 m**, fixing effective GSD at **2.93 cm/px**.	Load-bearing: matching training GSD to it was worth **+14.6 F1** in our measurements.
4	**Node spacing is 50 cm, not the documented 25 cm** (`min(min_length, 0.5)`).	Measured 0.47–0.50 m on real output across four orthomosaics.
5	**A transposed digit in the decoder** (265 filters where the symmetric ladder gives 256).	The published `.hdf5` is 31,140,642 parameters; the described architecture gives 31,036,673. Not reproducible from the paper without the typo.

Plus two coordinate traps in the vectoriser that produce plausible geometry rather than an error — the skeletonisation pad is never removed, and the EDT diameter branch reads pixel `(row, col)` as easting/northing.

Full detail with file and line references in Part II.

A reproduction worth noting. Trained single-stage on the published SpecDS and scored on the published TestDS with our own metric code: **0.7581**, which matches the 2022 paper's *best two-stage pretrained* result (75.6%) without any GenDS pretraining. The published beech model scores **0.6812** on that same TestDS — below the paper's figure, unexplained, and worth raising with the author.

1.2 Part II — Scale augmentation

The Analyzer's effective resolution is `tile_size / 512`, a user-set knob most people never touch deliberately. Jittering the training footprint ±30% makes a model robust to it:

	fixed spread	jitter spread	improvement	cost at 1.00×
HRNet, within-site	0.0295	0.0207	−0.0087 (t −6.0)	−0.0011 (noise)
UNet, within-site	0.0334	0.0146	−0.0188 (t −57.0)	**+0.0020** (3/3 seeds)
UNet, clean site holdout	0.1231	0.1116	−0.0115	**+0.0328** (3/3 seeds)

Recommendation: enable it by default. It costs nothing at the deployment scale and removes 30–56% of the degradation when the tile size moves.

On the one clean site holdout it also *gained* accuracy — but treat that number with care: the per-seed differences were **+1.1, +3.3 and +5.5 F1** (mean +3.3, 3/3 positive, but |t| = 2.6, **below the |t| ≥ 3 bar used elsewhere in this report**). One plot, one architecture, three seeds. The direction is consistent; the magnitude is not well determined, and it should not be quoted as "+3.3" without that range.

It does **not** replace matching the training scale to the serving scale — that is worth 14.6 F1 against this 0.9–3.3. Do both.

1.3 Part IV — Deployed on the Tegel surveys

Both orthomosaics (16.3 and 21.6 km²) run end to end through the Analyzer with our model and with the published one: **8,323 / 25,238 stems** against the published model's **6,483 / 20,141**, plus a diameter every 50 cm along each stem. Restricted to the frozen test plots the full run reproduces the plot-level scores to within 0.5 F1, so the pipeline is consistent from tile to full survey.

Counts alone are not evidence of quality — outside the five annotated plots nothing was digitised, so extra detections there cannot be judged.

1.4 Part III — Bachsee_north is not a bad site

Every model trained here scores F1 < 0.13 on Bachsee_north, and the corpus notes have long attributed this to its amber November canopy. Three explanations were tested; the first two are wrong.

checked	result
Are the labels misregistered? Annotations are EPSG:25833, ortho EPSG:32633 — a datum pair diverging ~0.5 m.	**No.** On the orthomosaic's own grid every beech site peaks at **zero** offset; residual under 4 cm.
Are the stems invisible under closed canopy?	**No.** That came from a *luminance-only* statistic. In CIELAB, Bachsee's stem/background separation is **1.75** — second highest in the corpus, above Campus's **1.16**, and Campus scores F1 0.53.
Is the site learnable at all?	**Yes.** Trained on 252 tiles from its own west half and tested on the spatially separated east half: **F1 0.6203**.

252 of its own tiles beat 3,588 tiles from every other site by 53 F1 points. The data is sound; the failure is **domain shift**.

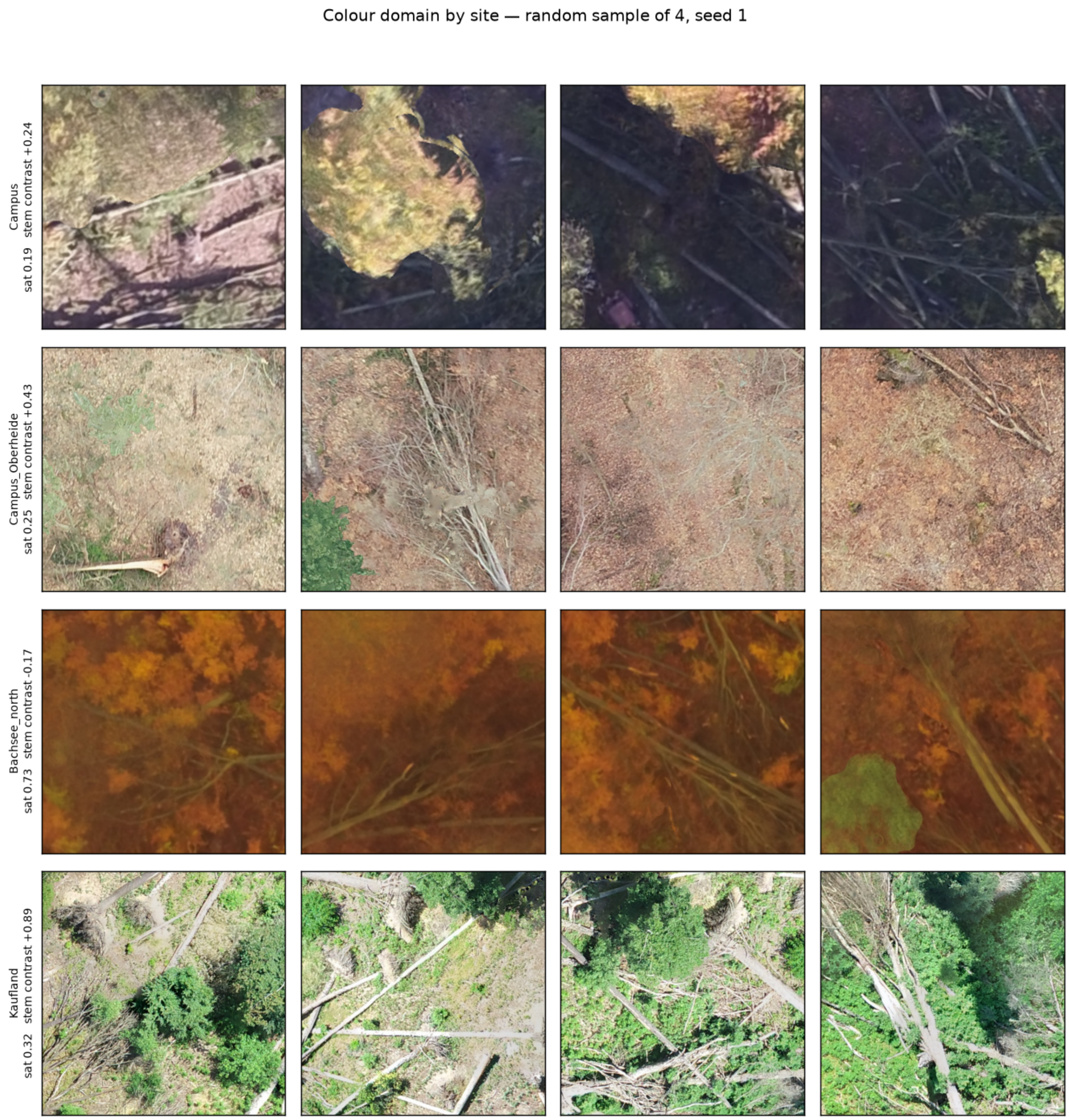


Figure 1: Colour domain by site — Bachsee_north is the only late-autumn acquisition

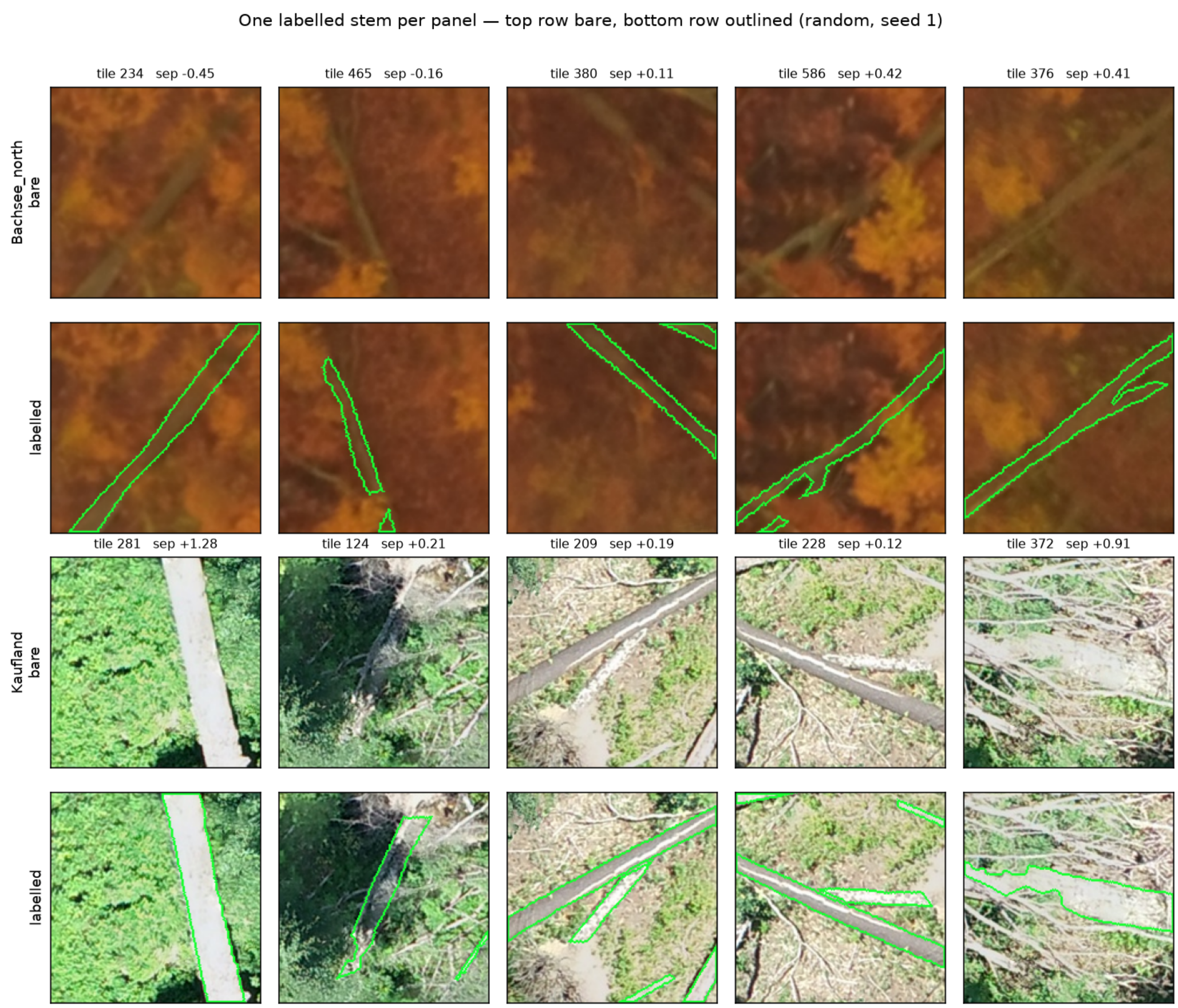


Figure 2: One labelled stem per panel: bare, then outlined. Bachsee's stems are visible

Re-running the folds with Bachsee **excluded from training** (18 runs) confirms it belongs there: overall **-0.045 F1**, 5 of 18 paired comparisons positive. The damage is ordered by phenological distance from Bachsee's November amber — Campus (October) **-0.095**, Campus_Oberheide (leaf-off February) **-0.045**, Kaufland (high summer) **+0.005** — and the two Campus folds lose the identical 800 tiles, so tile count does not explain the ordering. The out-of-domain site is the corpus's phenological diversity, not noise. Bachsee is the corpus's only autumn-phenology site, so when it is held out nothing in training has shown the model a stem against orange foliage. **Strong hue augmentation fixes it.** Randomising hue over the full circle (`--aug-hue-shift 90 --aug-sat-shift 60 --aug-hsv-p 0.9`) takes the Bachsee holdout from **F1 0.042 to 0.688** (3/3 seeds) for **−1.7 F1** in domain (Kaufland, 3/3 seeds). The base arm predicts almost nothing there — precision 0.87–1.00, recall 0.00–0.03; the strong arm reaches recall 0.58–0.66. This retires the standing advice that a site holdout is impossible on this corpus. It is an **interaction, not a dose**: a full hue circle at the default probability reaches only 0.261, and half the hue range at full probability only 0.177, while both together reach 0.688. Do not soften the setting.

An earlier claim in this repo that fold F1 is monotone in stem contrast came from the luminance-only statistic and is **withdrawn**.

1.5 What is not settled

- **The site holdout rests on one informative fold.** Of four beech sites, two are the same forest 4.4 years apart (33% footprint overlap) and one is Bachsee_north, which is outside every training domain (below).
- **Label scarcity.** Under 1 ha of stem is labelled in total; beech is near exhausted and pine has none.

2 Data inventory

Every number here was measured from the files on 2026-08-14, not transcribed from earlier notes. Rasters read with `rasterio`, vectors with `fiona`; areas are in each layer's own CRS units (m²). Source tree: `/Volumes/storage/Datasets/Winmol/training_data/WINMOL_Trainings_Data/GIS` (mounted on T14 as `/data/mnt/storage/...`).

2.1 The headline number

1.18 ha of labelled stem exists in total — 0.98 ha across 8 annotated sites in the main GIS tree, plus 0.20 ha from the Tegel Revier 12/13 surveys (section 6). 31 orthomosaics are held; **21 of them have no stem annotations at all**. Labelling, not imagery, is the binding constraint on this project.

2.2 1. Orthomosaics — all 29

Year is the acquisition date encoded in the filename (`YYYYMMDD`).

2.2.1 Beech (10 rasters)

acquisition	site	GSD	size	CRS
2017-10-16	EW_WW_Campus	6.39 cm	147 MPx	EPSG:32633
2017-11-14	EW_WW_Bachsee_north	4.08 cm	214 MPx	EPSG:32633
2017-11-23	EW_WW_Bachsee_south	3.95 cm	257 MPx	EPSG:32633
2021-07-06	EW_WW_Kaufland	**2.09 cm**	125 MPx	EPSG:25833
2022-02-26	EW_WW_Campus_Oberheide	3.36 cm	**1911 MPx**	EPSG:25833
2022-02-26	EW_WW_Campus_Oberheide (clip)	3.36 cm	373 MPx	EPSG:25833
2023-11-06/07	Altmann_Jauerling (AT)	3.30 cm	540 MPx	EPSG:3416
2024-07-19	FR17203_Abt-128_129_30	3.52 cm	280 MPx	**none**
2025-03-06	172_03_Kohlleiten_Meislingeramt (AT)	3.52 cm	280 MPx	EPSG:3416
2025-03-27	171_4_Kraking_Windwurf	3.99 cm	76 MPx	EPSG:25833

2.2.2 Spruce (13 rasters)

acquisition	site	GSD	size	CRS
2022-02-09	Bremerhagen 1–8	1.23 – 1.84 cm	302 – 802 MPx	EPSG:25833
2022-02-12	Barnekow 1, 3, 5, 6	1.51 – 1.76 cm	297 – 870 MPx	EPSG:25833
2022-02-12	Barnekow 2, 4 (**IR**)	1.75 / 2.07 cm	70 / 141 MPx	EPSG:25833
2022-03-09	EW_WW_Kaltes_Wasser_2	1.64 cm	815 MPx	EPSG:25833
2022-03-09	EW_WW_Ruheforst	1.61 cm	796 MPx	EPSG:25833

2.2.3 Pine (3 rasters)

acquisition	site	GSD	size	CRS
2022-03-09	EW_WW_Stromtrasse	2.75 cm	625 MPx	EPSG:25833
2022-03-29	Alt_Madlitz_plot123	2.58 cm	603 MPx	EPSG:25833
2022-03-29	Alt_Madlitz_plot4	3.24 cm	134 MPx	EPSG:25833

GSD spans 1.23 – 6.39 cm/px, a 5.2× range. The Analyzer serves at a fixed `tile_size / 512` (2.93 cm/px by default), so most of this corpus is nowhere near the serving scale unless resampled — which is what `--extent`/`--tile-px` extraction does.

2.3 2. Stem annotations — only 8 sites carry them

Units are in the header rather than every cell, and dates are in section 1. All layers are `EPSG:25833` **except Campus, which is 32633**.

site	species	polygons	stem m²	AOI m²
Campus	beech	822	2,908	118,808
Bachsee_north	beech	136	451	31,527
Kaufland	mixed	125	250	5,393
Campus_Oberheide	**mixed**	490	1,917	111,631
Bremerhagen_3	spruce	158	180	2,566
Barnekow_3	spruce/pine	358	741	15,516
Barnekow_5	spruce/pine	1,012	1,826	27,788
Barnekow_6	spruce	741	1,528	*none*
total		**3,842**	**9,802** = 0.98 ha	

A ninth file, `202171114_EW_WW_Bachsee_north`, is a **typo'd duplicate** of the 2017-11-14 layer — the same 136 polygons and 451 m². It has no matching orthomosaic. Ignore it; do not "fix" the name and pair it, or those stems are counted twice.

2.3.1 Species composition of the labelled polygons

code	species	polygons
GFI	Gemeine Fichte (Norway spruce)	1,986
RBU	Rotbuche (European beech)	1,211
GKI	Gemeine Kiefer (Scots pine)	231
DGL	Douglasie (Douglas fir)	97
GBI	Gemeine Birke (birch)	10
RGU	Roterle/other	1
—	no species attribute	~295

No pine site is annotated. The 231 GKI polygons are pine stems inside the two Barnekow spruce stands, not a pine acquisition — all three pine orthomosaics have zero labels.

"Beech" sites are not pure beech. Campus_Oberheide is GFI 199 / RBU 190 / DGL 92, so the beech models in this repo are trained on a mixed stand that happens to sit in the beech folder. Kaufland is RBU 110 / GBI 10 / DGL 5.

2.4 3. The four beech sites used for training

These are the sites behind every result in `scale-augmentation-results.md` and `scale-augmentation-loso.md`.

site	date	season	GSD	hue	saturation	notes
Campus	2017-10-16	autumn	6.39 cm	15°	0.19	coarsest raster in the corpus
Bachsee_north	2017-11-14	**late autumn**	4.08 cm	25°	**0.73**	amber canopy; CRS mismatch vs its ortho
Campus_Oberheide	2022-02-26	leaf-off	3.36 cm	32°	0.25	33% footprint overlap with Campus
Kaufland	2021-07-06	**high summer**	2.09 cm	98°	0.32	finest raster; smallest AOI

Two facts that shaped the experiments:

- **Campus and Campus_Oberheide are the same forest**, imaged 4.4 years apart; 33% of Campus's tiles overlap a Campus_Oberheide footprint (centroids 183 m apart). They are not independent sites, so a "leave-one-site-out" fold on either is really a temporal holdout.
- **Bachsee_north is the only late-autumn acquisition**, at 2.4–3.8× the saturation of every other site. Held out it scores F1 ~0.09; trained on its own west half it reaches **0.62** on its east half. Removing it from training costs the other folds up to 9.5 F1.

2.5 4. Published datasets (Zenodo record 5682580, Reder et al.)

Verified byte-identical against the record.

dataset	tiles	tile size	on disk	role
GenDS10	4,540	2000×2000	2.3 GB	synthetic stage-1 pretraining
SpecDS	454	313×313	12 MB	species-specific stage-2
TestDS	117	313×313	3.2 MB	the papers' test set

Two caveats measured here. **GenDS10's 2000 px tiles carry roughly 500–650 px of real information** — a round trip through 512 costs 1.45 grey levels against SpecDS's 3.73, so they are largely interpolation; effective GSD ≈ 4.6 cm/px. And **TestDS contains 117 tiles, not the 106 stems the 2022 paper describes**.

`GenDS10_512` (4,540 tiles, 736 MB) is our pre-resized copy: the loader's exact resize applied once instead of every epoch, verified to reproduce the loader's array to 0.0000/255.

2.6 5. Datasets built in this repo

All carry `tiles.jsonl` (source ortho, world centre, rotation, GSD, stem fraction), which is what makes any tile traceable back to the ground via `locate_tile.py` (private helper repo).

dataset	tiles	tile px	extent	GSD	purpose
BeechScale666	3,388 / 400 / 400	666	19.512 m	2.9297 cm	scale-augmentation arms
loso/site_all/Campus	800	666	19.512 m	2.9297 cm	LOSO fold source
loso/site_all/Campus_Oberheide	800	666	"	"	"
loso/site_all/Bachsee_north	800	666	"	"	"
loso/site_all/Kaufland	**388**	666	"	"	"
loso/site_tv/{Campus,Oberheide}	1,600 / 300 each	666	"	"	block train/val

Kaufland yields only 388 tiles because its AOI is 5,393 m² and a 19.512 m footprint holds 13.80 m in from every edge.

2.7 6. Tegel Revier 12 / 13 — added 2025-07, measured 2026-08-16

Two surveys held separately from the GIS tree above, digitised as five sample plots. Detail and results in `tegel-r12-r13-results.md`.

ortho	acquisition	GSD	size	CRS	bands
Revier 12 result_Res1.2_webp.tif	2025-07	**1.20 cm**	339k × 334k px, 12 GB	EPSG:32633	RGBA
Revier 13 result_Res1.3_COG.tif	2025-07	**1.28 cm**	393k × 335k px, 19 GB	EPSG:32633	RGB

plot	polygons	stem area	AOI
R12-P1	178	215 m²	14,913 m²
R12-P2	552	891 m²	23,080 m²
R12-P3	299	501 m²	8,692 m²
R13-P1	232	248 m²	8,865 m²
R13-P2	123	141 m²	13,436 m²
total	**1,384**	**1,996 m² = 0.20 ha**	

This raises the corpus from 0.98 ha to 1.18 ha of labelled stem, +20%, and adds the first substantial pine (153 polygons), birch (120) and ash (41) annotations. Labels are `EPSG:25833` against `EPSG:32633` orthos; registration verified at zero offset on all plots. Species names are free text with the same variant problem as the older corpus (`POPLAR` / `WHITE POPLAR` / `SILVER POPLAR`).

2.8 7. Defects to handle

- **Mixed CRS.** Campus and Bachsee north/south orthos are EPSG:32633; everything else is EPSG:25833 — the same UTM zone on different datums, so a silent mismatch shifts masks by metres rather than failing. `20171114_EW_WW_Bachsee_north` has annotations in 25833 and its ortho in 32633. Both scripts reproject explicitly, and this was **verified**: every beech site's polygons peak at zero offset on its own raster grid (`check_registration.py`, private helper repo), residual under 4 cm.
- **One orthomosaic has no CRS at all** (`20240719_FR17203_Abt-128_129_30.tif`). It is unannotated, so nothing depends on it yet.
- **`202171114_EW_WW_Bachsee_north`** — typo'd duplicate, see above.
- **Species case variants**: `RBU`/`rBU`/`RBu`, `GFI`/`GFi`/`gFI`, `DGL`/`DGl`/`dGL`, plus single-record typos `GIF` and `GFIU`. Match case-insensitively or silently drop records. ~295 polygons carry no species at all.
- **No DEM/DSM/CHM anywhere** in the corpus — height is not available as a channel.
- **Two Barnekow rasters are infrared** (2 and 4), not RGB; they are unannotated.
- **`gends10_ready`** is a locally derived copy that is not trusted; use the Zenodo originals.

3 Reder's method: gaps closed from source, and where the docs disagree with the code

The methodological report on Reder et al. 2022 / 2025 lists items "NOT recoverable from openly accessible sources" and points at `WINMOL_segmentor` (R) and `WINMOL_Analyzer` (Python) as the authoritative places to read them. Both were read directly. This closes what is closeable and flags three places where the published description and the code disagree.

Every value below is quoted from source or measured by running it, with the file and line.

3.1 1. Hyperparameters — closed

item	value	source
optimizer	Adam, `lr = 1e-3`, β_1 0.9, β_2 0.999, `decay = 0`, `amsgrad = FALSE`	`controlling.R`
batch size	**4**	`main_training.R`
epochs	100 cap per stage	`training.R`
early stopping	`val_loss`, patience **3** (stage 1) / **5** (stage 2)	`callbacks.R:15-17`
LR schedule	ReduceLROnPlateau, factor 0.1, patience 2, `min_delta = 1e-4`	`callbacks.R:7-12`
checkpoint	`save_best_only = TRUE` on `val_loss`	`callbacks.R:6`
augmentation	flips + brightness/contrast/saturation/hue, train split only	`input_pipeline.R`

3.2 2. Split construction — closed, and it is not site-level

```
train_data <- initial_split(train_data, prop = 0.8)   # training.R, both stages
```

A **random 80/20 split over tiles**, drawn independently per stage. Not spatial, not site-level. The report infers a site-aware evaluation from the 15-site design; the training code does not implement one. With the generator's 100× oversampling, tiles overlap, so this validation signal is optimistic — which matters because early stopping, the LR schedule and checkpoint selection all read it.

3.3 3. The composite loss does not train — measured

Both papers present the F1/BCE composite as the key choice for foreground/background imbalance. In code:

```
F1Score <- custom_metric("F1Score", function(y_true, y_pred) {
  y_pred <- k_round(y_pred)          # <- hard threshold
  ... })
F1Score_loss <- function(y_true, y_pred) {
  loss_binary_crossentropy(y_true, y_pred) + (1 - F1Score(y_true, y_pred)) }
```

`F1Score_loss` calls the **metric**, and the metric rounds. `k_round` has zero derivative almost everywhere, so the F1 term contributes no gradient: **the model trains on plain binary cross-entropy.** The term is real in the reported loss value and absent from the optimisation.

Verified three independent ways:

- **In torch** — R's expression transcribed gives a gradient `torch.equal` to plain BCE's (grad-norm 0.012018 for both), while the reported value is ~2× larger.
- **In R itself** — `LOSS=f1` (Stefan's untouched `controlling.R`) against `loss_binary_crossentropy` alone, same seed, 8 epochs: precision, recall and F1 agree to **eight decimal places** at every epoch. Only `loss` differs, by exactly `1 - F1`.
- **Ablated** — replacing it with a *differentiable* soft-F1 term, paired over seeds, shifts recall **+3.8** and precision **–2.5** (4/4 seeds, paired t 4.86 / –5.47). So the term does something once it has a gradient; in the published form it does nothing.

One important qualification (re-verified 2026-08-19). "The term does nothing" is too strong. It contributes no *gradient*, so the optimisation is pure BCE — confirmed again in torch, where the composite and plain-BCE gradients are bit-identical (max difference exactly 0.0), while a *differentiable* soft-F1 term does change them (2.6e-4), so the test is sensitive enough to detect a real difference.

But the composite **value** is what Keras monitors: `save_best_only` on `val_loss`, `ReduceLROnPlateau` on `val_loss`, and early stopping all read `BCE + (1 - F1)`. That ranks epochs differently from BCE alone, so **which checkpoint is kept and when the learning rate drops can differ**, even though every training step is identical. The term is inert in the optimiser and active in model selection.

This does not invalidate the results — BCE is a reasonable loss — but the stated rationale for the loss choice is not what the model experienced.

3.4 4. Architecture — a transposed digit

`model_UNet.R:87,91`, decoder stage E7:

```
layer_conv_2d(filters = 265, ...)    # every other stage is symmetric: 512, 256, 128, 64
```

The transpose conv above it correctly emits 256. Both convs of that block use **265**. The published `.hdf5` loads at **31,140,642** parameters; the symmetric ladder gives 31,036,673. Harmless numerically, but "31M U-Net" is not reproducible from the paper's description without the typo.

3.5 5. Instance-extraction defaults — closed

`classes/Config.py`:

parameter	default	GUI label in the report
`min_length`	**2.0** m	Min Length
`max_distance`	**8**	Max Distance
`tolerance_angle`	**7**	Maximum Angle
`max_tree_height`	**32** m	Maximum Tree Height
`tile_size`	**15** m	Tile side length
`img_width`	**512** px	Tile pixel extent
`overlap_pred`	8	—
`stem_binary_threshold`	0.5	—

Tile side length is 15 m, not ≈15.4 m. The report derives 15.36 m from 3 cm × 512 px; the code fixes 15 m, giving an effective GSD of **15/512 = 2.93 cm/px**. That number is load-bearing: matching training GSD to it was worth **+14.6 F1** in our own measurements.

Skeletonisation (`utils/Skeletonization.py`): `skimage.morphology.skeletonize` on the binary mask, after padding by `int(max_tree_height / px_size) + 1`. Endpoints and branchpoints are found by crossing-number (transitions `==` 1 endpoint, ≥ 3 branchpoint); branchpoints are removed, dense nodes eroded, then segments refined to the measuring-point spacing with a minimum length of `floor((min_length/4) / px_size)`.

Diameter: two methods, `diameter_method = "contour"` (default) or `"edt"`, measured on a local normal (`_local_normal`, `_measurement_vector`, `diameter_vector_half_length_m = 1.0`) — so "perpendicular to the centerline" is confirmed.

Volume: `calc_l_v(p1, p2, d1, d2)`, truncated cone between consecutive nodes. Confirmed.

3.6 6. Node spacing is 50 cm, not 25 cm

The WINMOL documentation says the diameter is determined "every 25 cm". The code says:

```
measuring_point_spacing_m = 0.5                          # Config.py:104
measuring_point_spacing = math.floor(
    min(config.min_length, config.measuring_point_spacing_m) / px_size)   # Skeletonization.py
```

`min(2.0, 0.5) = 0.5` -> **50 cm**. Measured on real output across four orthomosaics, the mean gap between consecutive centreline vertices is **0.47–0.50 m**. Either the docs describe a different version or the figure is wrong; the shipped default is 50 cm.

3.7 7. Two coordinate traps in the vectoriser

Re-checked 2026-08-19 against the current `winmol-oom-fix` version: both are still present. Both produce plausible geometry rather than an error, so they are invisible without checking:

- **The skeletonisation pad is never removed.** `find_segments` pads by `max_tree_height / px_size` and returns coordinates still carrying that offset — 1530 px at 2.09 cm/px. The comment at `Skeletonization.py:175` claims the offset is removed; `np.argwhere` returns raw padded indices.
- **Paths are pixel `(row, col)`, not world coordinates**, and `path.length` is a pixel count. World coordinates are applied only at write time. `calc_v_d_edt` passes `stem.path.coords` to `_xy_to_rowcol`, which reads them as easting/northing — so the **EDT diameter branch appears broken**; the default contour path is self-consistent in pixel space. The current version adds a bounds check returning `radius = 0.0` when the bogus index falls outside the map, so the bug now fails **silently as zero diameters** rather than raising — harder to notice, not easier.

3.8 8. CLI: `Stems` does not vectorise

`run_stem_pipeline` calls the prediction phase and stops; only `run_tree_pipeline` (`Trees`) runs `run_vector_phase`. A `Stems` run exits cleanly with a stem raster and no vectors — which reads as a failure and is not one.

3.9 What this changes in the report's recommendations

Stage 1 (data). Target the code's actual scale — 15 m / 512 px = 2.93 cm/px — rather than "3 cm". Our measurement: serving a model at a GSD it was not trained on cost 14.6 F1, and the same published beech model scores **0.8003** on native-512 tiles versus **0.6812** on Reder's own 313 px TestDS (upsampled ×1.64 to reach the input). Resolution handling, not model quality, explains a 12-point swing.

Stage 2 (splits). The report's advice to use site-level holdout is right and the code does not do it. Worth knowing what it costs: in our corpus a whole-site holdout costs 8–11 F1, and one site (November beech canopy, 100% amber pixels) returns F1 **exactly 0.0000** — so a leave-one-site-out mean can be carried entirely by one dead fold. Report per-fold.

Stage 2 (copy-paste + two-stage). GenDS tiles are 2000×2000 but carry ~500–650 px of real information — round-trip loss through 512 is 1.45 grey levels at 2× against SpecDS's 3.73, and edge energy is 7.97/px against 15.52. Effective GSD ≈ 4.6 cm/px, which matches SpecDS's 4.8 cm/px, so the two stages are well aligned with each other but not with the Analyzer default. On our own corpus, synthetic pretraining measured **−0.2 to −1.4** once a validation-split bug was fixed (it had read +1.1 to +3.6 before).

Stage 4 (evaluation). Pixel F1 is not comparable to DR25 — different quantity. Also fix a noise floor before comparing anything: two runs with **bit-identical gradients** landed 2.0 F1 apart without deterministic kernels, 0.5–1.0 with. Single-run differences below that are unreadable, which is why the ablations above are paired over 5 seeds.

3.10 A reproduction worth noting

The 2022 paper reports pixel F1 **72.6%** for SpecDS-only training and **75.6%** for its best two-stage pretrained model, on TestDS.

Trained single-stage on the same published SpecDS, scored on the same published TestDS with our own metric code: **0.7581**. That matches the paper's *best pretrained* result without any GenDS pretraining. The R U-Net retrained under the same conditions gives 0.7424.

Two caveats. The published **beech** model (`SpecDS_Beech_512`, 2023-02-28) scores only **0.6812** on that same TestDS — below the 2022 paper's figure, unexplained, and worth raising with the author. And TestDS as distributed on Zenodo contains **117** tiles, not the 106 stems the paper describes (verified by md5 against the record).

Sources read directly: `WINMOL_segmentor` — `controlling.R`, `training.R`, `callbacks.R`, `main_training.R`, `model_UNet.R`, `input_pipeline.R`. `WINMOL_Analyzer` — `Config.py`, `Skeletonization.py`, `Vectorization.py`, `Quantification.py`, `winmol_run.py`. Datasets verified byte-identical against Zenodo record 5682580.

4 Scale augmentation: ±30% footprint jitter buys robustness for free

Design and yardstick were fixed before training — pre-registered in `specs/2026-08-13-scale-augmentation-design.md`, which now lives in the private helper repo alongside the tooling below. Numbers below are copied verbatim from `assets/scale-sweep.json`, rendered by `plot_scale_sweep.py` (private helper repo) from the six per-model sweeps in `assets/scale-aug/`. The sweeps themselves were cut by `scale_sweep.py`, same repo.

Answer: ship it. Measured on two architectures. On HRNet, jitter flattens the scale curve by **0.87 F1** (3/3 seeds, paired t −6.02) and costs **0.11 F1** at the native scale — 1/3 seeds positive, i.e. noise. On UNet the same manipulation flattens it by **1.88 F1** (3/3 seeds, t −56.95) and *gains* 0.20 F1 at native scale as well. It wins at every scale on every UNet seed.

Sections below cover HRNet first, then the UNet replication and a reproducibility check.

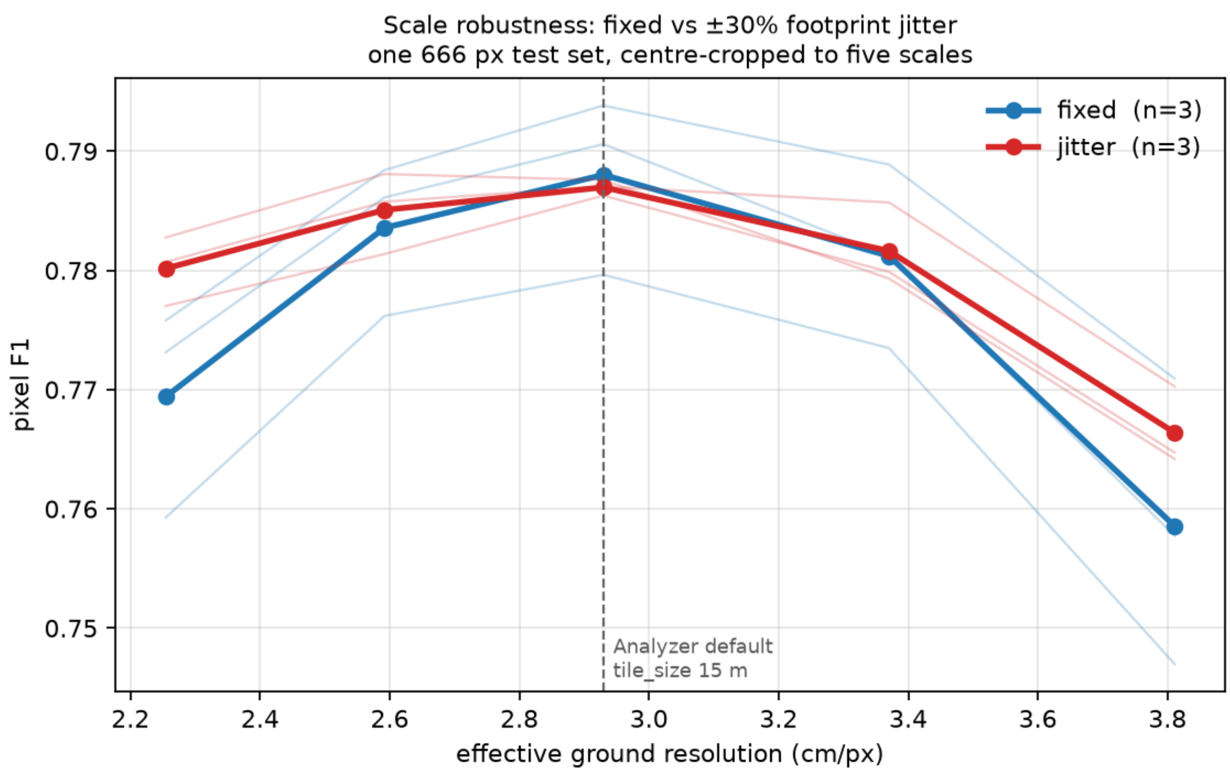


Figure 3: F1 versus effective ground resolution

4.1 Setup

data	`BeechScale666` — 3388 / 400 / 400 tiles at 666 px, 19.512 m, **2.9297 cm/px**
sites	Campus + Oberheide block-split (60 m); Bachsee_north, Kaufland whole-to-train
leak check	closest train<->val<->test approach **28.3 m** / **27.9 m** vs a **27.59 m** floor
arms	crop 512–512 (*fixed*) vs 394–666 (*jitter*), `RandomSizedCrop->512`
held fixed	same tiles, rotation/flip/photometric, arch, batch 16, `--deterministic`
n	3 paired seeds per arch (HRNet, UNet); test = 400 tiles, CPU EP, threshold 0.5

Both arms train on **the same 666 px source tiles** and both get random crop *position* — only the scale distribution differs. That is what makes the comparison clean.

4.2 HRNet: per-scale result

Differences are jitter – fixed, paired within seed.

crop	ratio	GSD	fixed	jitter	diff	sign	t
394	0.77×	2.254	0.7694	0.7802	**+0.0108**	3/3	2.87
453	0.88×	2.592	0.7836	0.7851	+0.0015	2/3	0.66
512	**1.00×**	**2.930**	0.7880	0.7870	−0.0011	1/3	−0.26
589	1.15×	3.370	0.7812	0.7816	+0.0004	1/3	0.14
666	1.30×	3.811	0.7586	0.7664	**+0.0078**	2/3	1.46

Drop from each arm's own peak — the robustness claim, independent of absolute level:

arm	s1	s2	s3	mean
fixed	0.0229	0.0326	0.0328	**0.0295**
jitter	0.0168	0.0215	0.0239	**0.0207**

Paired: **−0.0087, 3/3 seeds flatter, t −6.02.**

4.3 Reading it

The win is at the flanks and it is a recall win. At 1.30× the fixed arm's precision *rises* to 0.819–0.829 while recall falls to 0.680–0.728: coarser pixels make stems thinner, the model stops committing, and it under-segments. This is the benign failure mode — an under-reporting model, invisible in F1 alone. Jitter holds recall 3–6 points higher there at some precision cost. Every domain-shift failure measured in this project has had this same shape.

The per-scale t-values are weak; the spread test is not. With n=3, only the 0.77× point approaches significance on its own. But the spread statistic uses each seed's whole curve rather than one point of it, and it is unanimous with |t| = 6.0. The per-scale rows are best read as *where* the effect lives, not as five independent tests.

The cost at native scale is nil. −0.0011 F1, 1/3 seeds positive. The worry going in — that spreading training over 2.25–3.81 cm/px would blunt the model at the one resolution it is actually served at — did not materialise. Note also that validation ran at 1.00× (`--eval-tiling`), which structurally favours the fixed arm, so this is if anything generous to the baseline.

4.4 Replication on UNet — the effect is real and larger

Repeated identically with `--arch unet` (same dataset, crop ranges, seeds, everything); sweeps in `assets/scale-aug-unet/`, aggregate in `assets/scale-sweep-unet.json`.

crop	ratio	GSD	fixed	jitter	diff	sign	t
394	0.77×	2.254	0.7492	0.7689	**+0.0197**	3/3	14.97
453	0.88×	2.592	0.7760	0.7802	+0.0042	3/3	3.67
512	**1.00×**	**2.930**	0.7812	0.7833	+0.0020	3/3	1.82
589	1.15×	3.370	0.7729	0.7792	+0.0063	3/3	3.97
666	1.30×	3.811	0.7498	0.7690	**+0.0191**	3/3	6.13

Spread: fixed **0.0334** (0.0344/0.0333/0.0326) vs jitter **0.0146** (0.0159/0.0138/0.0142). Paired: **−0.0188, 3/3 seeds flatter, t −56.95.**

Jitter wins at **every** scale on **every** seed, and unlike HRNet it does not even cost anything at 1.00× — it gains 0.20 F1 there, 3/3 seeds.

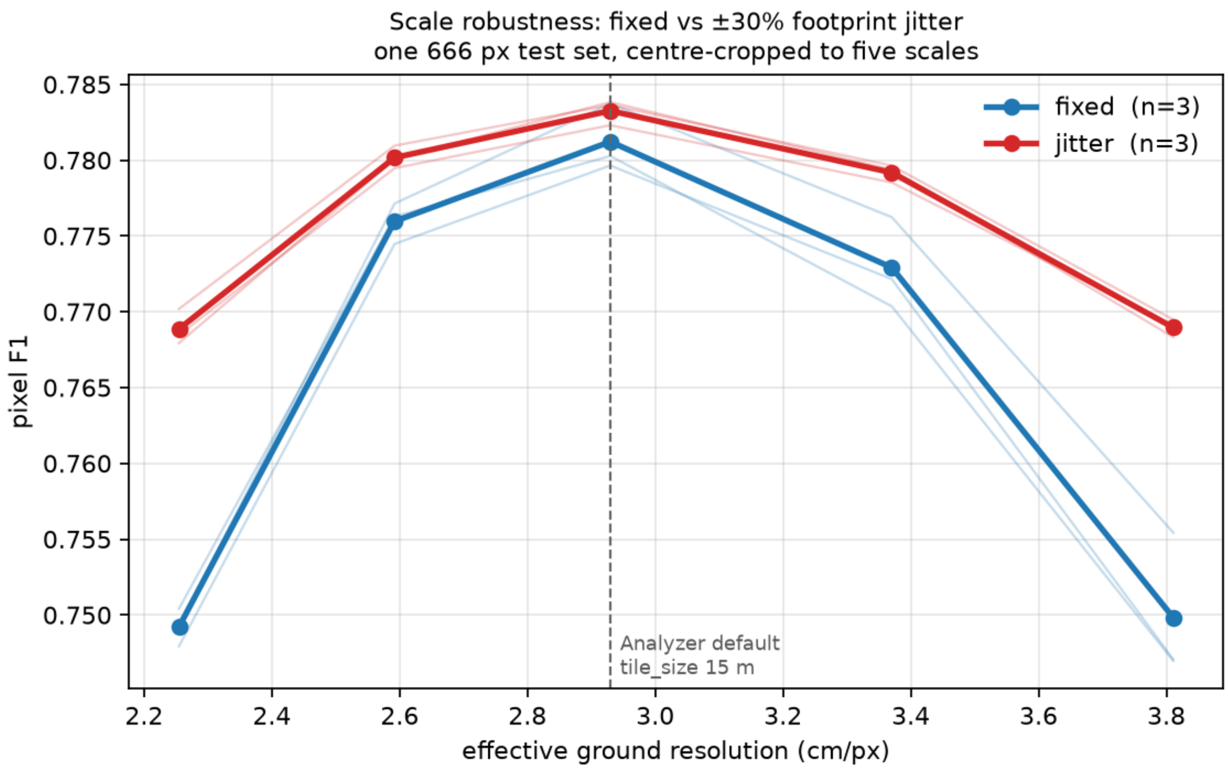


Figure 4: UNet: F1 versus effective ground resolution

Why UNet gains more. HRNet carries parallel branches at several resolutions through the whole network and fuses them repeatedly, so multi-scale context is architectural. A plain UNet has one resolution ladder and must *learn* scale invariance from the data. That predicts exactly what is measured: UNet is the more scale-brittle baseline (spread 0.0334 vs HRNet's 0.0295) and gains the most from jitter (–0.0188 vs –0.0087). The two results agree in direction and differ in magnitude in the direction the architectures predict, which is a stronger joint result than either alone.

	fixed spread	jitter spread	gain	cost at 1.00×
HRNet	0.0295	0.0207	−0.0087 (t −6.0)	−0.0011 (1/3, noise)
UNet	0.0334	0.0146	−0.0188 (t −57.0)	**+0.0020** (3/3)

4.5 Reproducibility check

`fixed-s1` scored highest in its arm in both architectures, so it was re-run from scratch with the same seed on a different GPU:

	F1	precision	recall
original (GPU 0)	0.7922	0.7990	0.7856
reproduction (GPU 7)	0.7922	0.7990	0.7856

Those are `test_results.md` figures — 1600 windows from `--eval-tiling`, not the sweep's 400 centre crops, so they differ slightly from the 1.00× column above (0.7922 vs 0.7938 for the same model). Every arm-vs-arm number in this report comes from the sweep; the table here is a run-identity check, not a scale measurement.

Identical to four decimals. `--deterministic` reproduces across devices, so between-seed differences of 1–2 F1 in the fixed arm are genuine seed variance, not run-to-run noise — and being common to both arms, they cancel in the paired differences. Runs now write `run_config.json` beside the model so this is auditable from the artifacts rather than requiring a re-run.

4.6 Caveats, stated

- **F1 is not comparable across scales.** A 23 cm stem is 10.2 px at 2.25 cm/px and 6.0 px at 3.81. The downward slope on the right of the figure is partly the task getting harder, not the models getting worse. Only the arm-vs-arm gap within a column is readable.
- **Arm B occasionally sees the full 666 px tile; arm A never does.** A wider crop range means a wider field of view. This is inherent to the manipulation and cannot be separated from scale jitter without also changing the footprint.
- **This is not the earlier "5.2 F1 across ±30% zoom" figure.** That came from a different model and a test built by re-sampling the orthomosaic per scale, which put each scale on different ground. The 2.95 F1 (HRNet) and 3.34 F1 (UNet) measured here are the clean version of the same quantity and supersede it; the two are not directly comparable.
- **Sites are shared across splits.** The claim is scale robustness within known forests, not transfer to a new site.
- **n = 3.** Enough for the unanimous spread result, thin for the per-scale rows.

4.7 Recommendation

Enable `--multiscale --crop-min-px 394 --crop-max-px 666` by default for beech training, on both architectures. It removes **30%** of the scale degradation on HRNet and **56%** on UNet, and costs nothing at `tile_size = 15` — on UNet it gains there too. Users do move the tile-size spinbox, and nothing in the model warns them when they have.

This does **not** replace matching the training scale to the serving scale. That was worth 14.6 F1; this is worth 0.9–1.9. Be robust *and* correctly scaled.

Two questions worth answering next, in order:

1. **Does a wider range keep paying?** The corpus spans 1.58–6.39 cm/px, a 4× range, against the 1.7× tested here. The UNet curve is still falling at both ends of the sweep, so ±30% is unlikely to be where the gain stops.
2. **Does it survive a site holdout?** ~~These splits share sites.~~ **Answered** — see `scale-augmentation-loso.md`. Flatter in 4/4 folds, and worth **+1.1 / +3.3 / +5.5 F1 across the three seeds** (mean +3.3, 3/3 positive, |t| = 2.6 — below the |t| >= 3 bar used elsewhere here, so the direction is reliable and the magnitude is not) on the one clean holdout where the model segments at all. Two of the four folds turned out not to be site holdouts (Campus and Campus_Oberheide are the same forest 4.4 years apart, 33% footprint overlap) and one was dead (Bachsee_north, F1 < 0.13 for every arm), so the new-site evidence is one strong fold rather than four.

5 Scale jitter under a site holdout — four folds, reported individually

Follow-up to `scale-augmentation-results.md`, which measured scale jitter on splits that **share sites**. This asks whether the robustness survives when the test site is absent from training. UNet, 3 paired seeds per arm per fold, 24 runs. Numbers copied verbatim from `assets/loso-*.json`.

Answer: it survives, but the evidence is thinner than the within-site result and one fold is uninformative. Jitter's scale curve is flatter in **4 of 4 folds**; only one fold reaches the usual significance bar on its own.

5.1 Two of these folds are not site holdouts

`tiles.jsonl` says **33% of Campus tiles overlap a Campus_Oberheide footprint** (centroids 183 m apart). They are the same forest imaged 2017-10-16 and 2022-02-26, and windthrown stems do not move between acquisitions — so those are largely the same physical stems.

fold	test tiles	nearest training site	what it measures
Bachsee_north	800	**5.68 km**	new-site transfer
Kaufland	388	**3.56 km**	new-site transfer
Campus	800	0 m (33% overlap)	4.4-year acquisition gap
Campus_Oberheide	800	0 m (30% overlap)	4.4-year acquisition gap

Reported under those two headings, never pooled.

5.2 Every fold, no mean

`spread` = each seed's peak-minus-worst F1 across 0.77× / 1.00× / 1.30×. Lower is more scale-robust; the paired column counts seeds where jitter is flatter.

fold	F1@1.00× fixed	jitter	spread fixed	spread jitter	flatter	t
Bachsee_north	0.0945	0.0434	0.0940	0.0778	1/3	−0.15
Kaufland	0.6737	**0.7065**	0.1231	0.1116	2/3	−1.31
Campus	0.5263	0.5290	0.1125	0.0886	2/3	−1.41
Campus_Oberheide	0.6793	0.6792	0.0408	**0.0272**	3/3	−5.52

Direction is unanimous — all four mean spreads favour jitter — but only Campus_Oberheide clears |t| ≥ 3 with unanimous seeds. State this as a fold count, not a mean: *flatter in 4/4 folds, individually significant in 1/4.*

5.2.1 Bachsee_north is a domain-shift fold, not a defective one

F1 **0.03–0.13** for every arm and seed. Three explanations were tried; the first two are wrong and are recorded here because each looked convincing.

Not a labelling error. Its annotations are EPSG:25833 and its orthomosaic EPSG:32633 — the same UTM zone on different datums, a pair that has diverged ~0.5 m through plate motion, and already logged as a known defect. Measured with `check_registration.py` (private helper repo) on the orthomosaic's **own unrotated grid**, every beech site peaks at zero offset:

site	stems / ortho CRS	contrast at zero	bright peak	verdict
Bachsee_north	25833 / 32633 **mismatch**	+0.118	(0.00, 0.00) m	registered
Campus	32633 / 32633	+0.384	(0.00, 0.00) m	registered
Campus_Oberheide	25833 / 25833	+0.811	(0.00, 0.00) m	registered
Kaufland	25833 / 25833	+0.797	(0.00, 0.02) m	registered

The pipeline's `transform_geom` applies the datum shift; residual under 4 cm.

Not invisible stems either — this was my error. The table above measures *luminance*, and on that basis Bachsee looked like a closed canopy hiding its stems. It is not. Its stems differ from the background in **colour**, not brightness. In CIELAB over 100 random tiles per site:

site	luminance sep	chroma sep	**full Lab sep**	fold F1
Kaufland	0.77	1.98	**2.14**	0.71
Bachsee_north	0.44	1.66	**1.75**	**0.09**
Campus_Oberheide	0.56	1.51	1.70	0.68
Campus	0.53	0.95	**1.16**	0.53

Bachsee has the **second-strongest** stem/background separation in the corpus. Campus has the weakest and scores 0.53. An earlier claim in this repo that fold F1 is monotone in stem contrast came from the luminance-only statistic and is **withdrawn**.

The close-ups make it plain — pale olive stems against orange canopy, labels sitting on them:

It is domain shift, and one training run proves it. Trained on 252 tiles from the west half of Bachsee and tested on the spatially separated east half (30.3 m closest approach against the 27.59 m floor):

training data	test	F1	precision	recall
3,588 tiles, other sites (the LOSO fold)	Bachsee	**0.09**	—	—
252 tiles, Bachsee's own west half	Bachsee east	**0.6203**	0.6061	0.6351

252 of its own tiles beat 3,588 of everyone else's by 53 F1 points. The signal is there, the labels are on it, and the site is learnable. It fails as a holdout because it is the corpus's **only autumn-phenology site**: held out, nothing in training has ever shown the model a stem against orange foliage.

Green is ground truth, red is prediction; on most tiles there is no red at all — recall collapse in its extreme form. The fold remains a **tie** for arm comparison, but for a different reason than "hard data": both arms are equally outside their training domain.

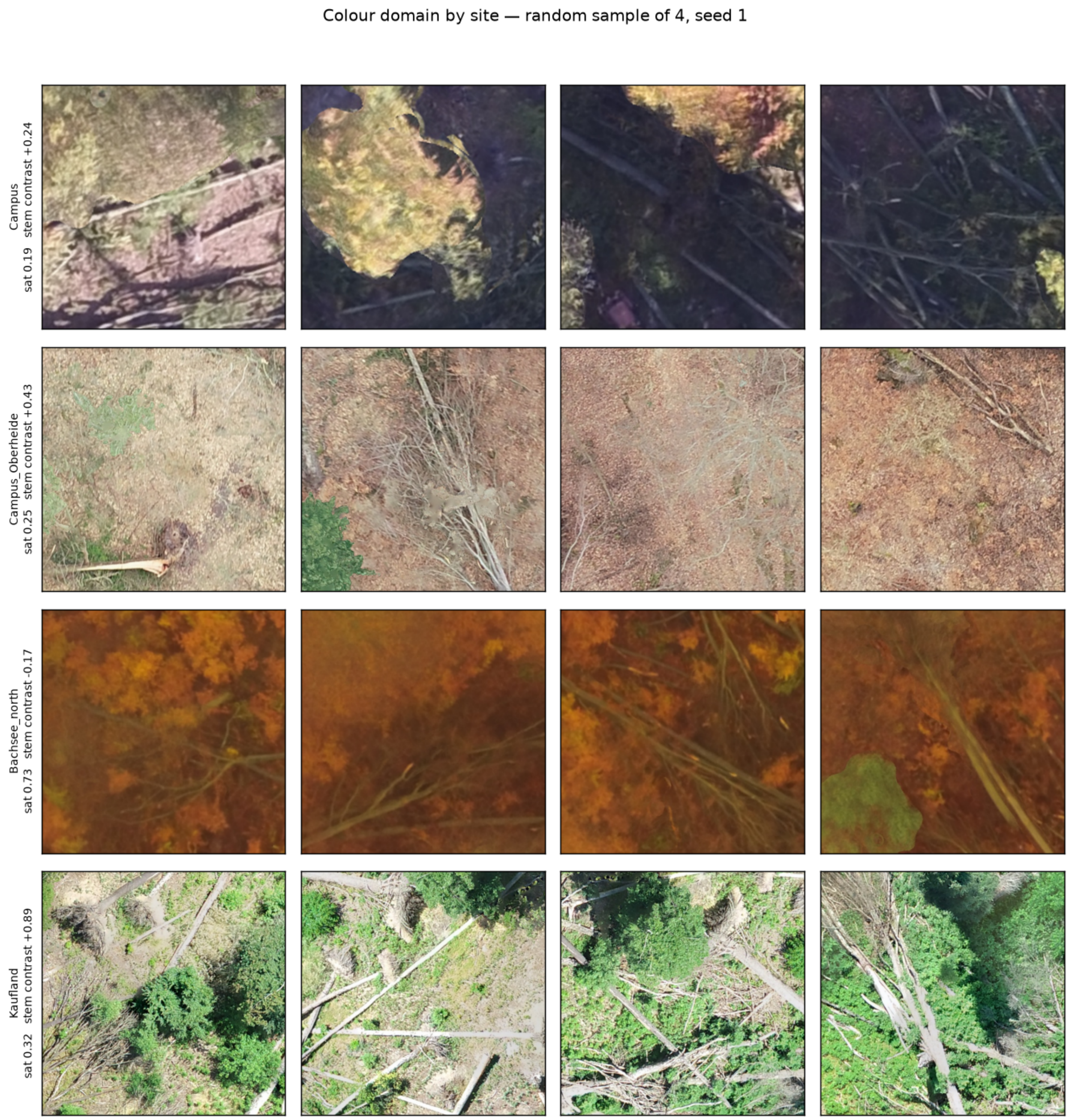


Figure 5: Colour domain by site

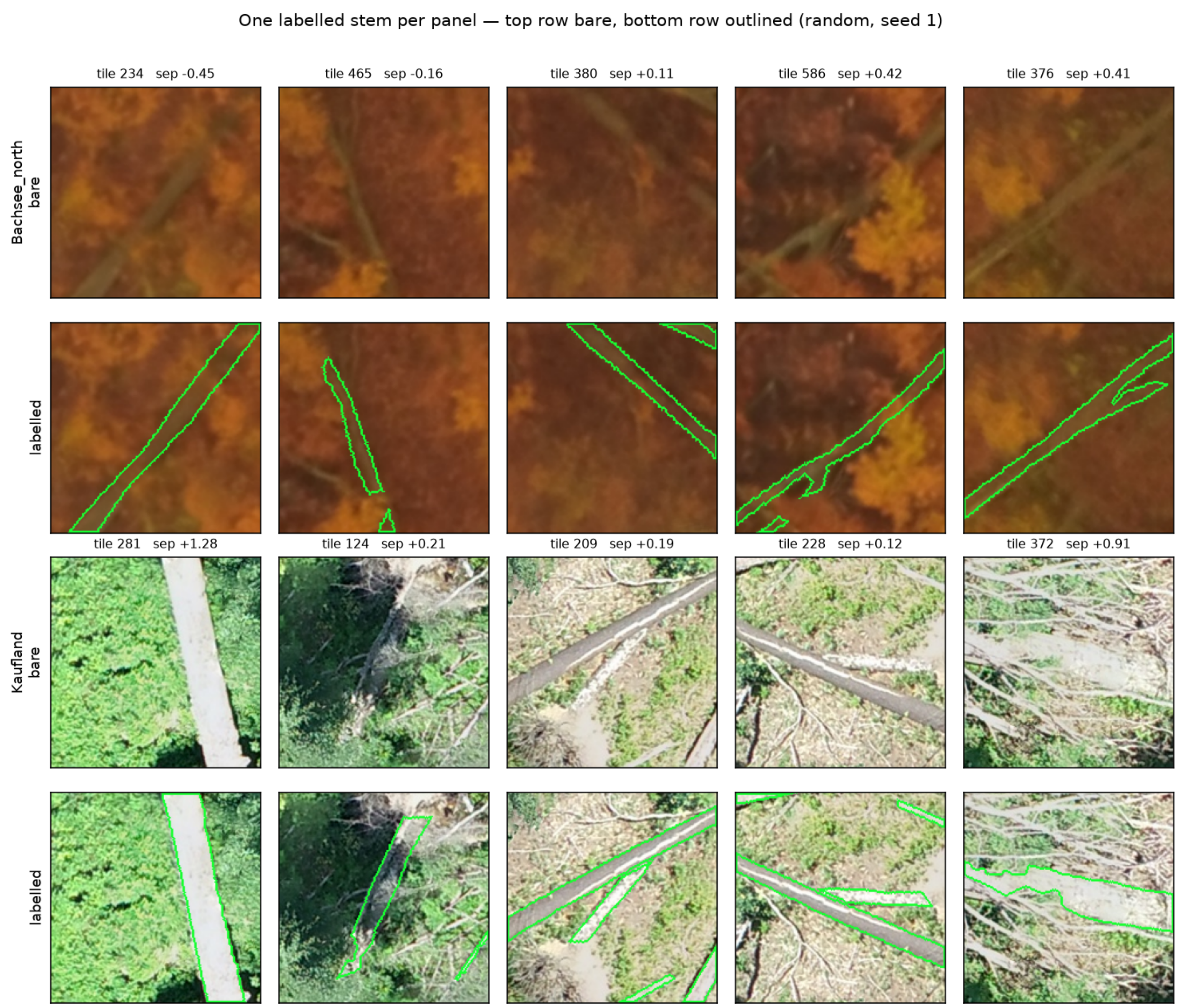


Figure 6: Bachsee stem close-ups

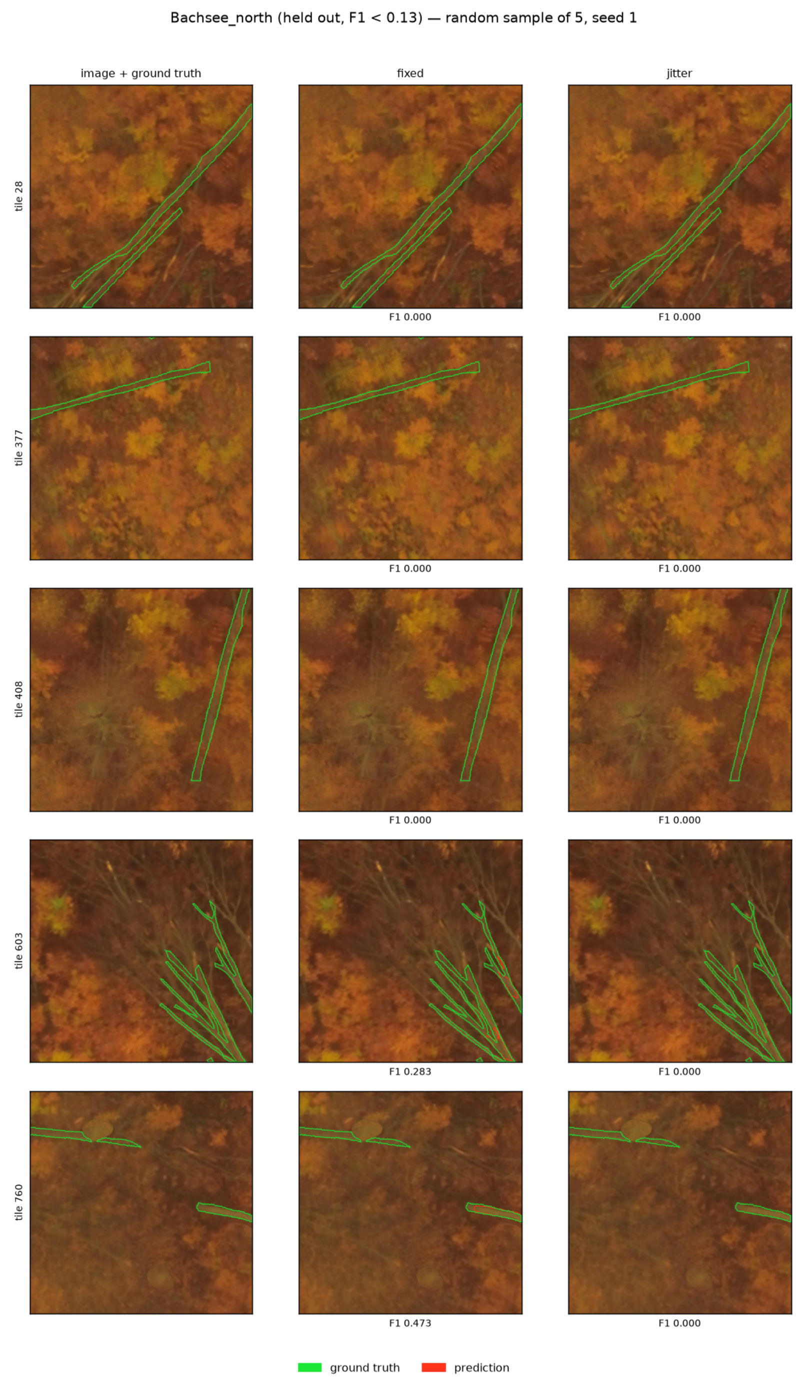


Figure 7: Bachsee_north examples

5.2.1.1 Strong hue augmentation fixes it — +64.6 F1

If Bachsee fails only because no training site is amber, randomising hue should remove the colour shortcut and force the model onto structure. Tested: 12 UNet runs, 3 paired seeds, arms differing **only** in `HueSaturationValue` limits, both re-run so nothing is inherited.

arm	hue	sat	val	p
base	±20	±30	±20	0.5
strong	**±90** (any hue)	±60	±30	0.9

fold	base	strong	delta	signs
Bachsee_north (out of domain)	0.042	**0.688**	**+0.646**	3/3
Kaufland (in domain)	0.708	0.690	−0.017	0/3

Per seed on Bachsee: 0.0000 / 0.0639 / 0.0619 -> 0.6660 / 0.7059 / 0.6916.

The failure mode inverts. The base arm's precision is 0.87–1.00 with **recall 0.00–0.03** — it predicts essentially nothing. Strong moves recall to 0.58–0.66 at precision 0.76–0.79. Note 0.688 also exceeds the 0.62 that training on Bachsee's *own* west half achieved, so the other three sites do contain the information; the model simply could not reach it through the colour gap.

Cost: −1.7 F1 in domain (3/3 seeds, |t| 1.6). That is a real cost and small enough to be worth paying for anything expected to meet a new acquisition or season.

5.2.1.2 It is an interaction, not a dose — milder settings fail

`base` and `strong` differ in four knobs at once, so two further arms decompose it (24 runs total, same 3 seeds):

arm	hue	sat	val	p	Bachsee	delta vs base	Kaufland	delta
base	±20	±30	±20	0.5	0.042	—	0.708	—
mid45	±45	±60	±30	0.9	0.177	+0.135	0.705	−0.002
hueonly	±90	±30	±20	0.5	0.261	+0.219	0.707	−0.001
strong	±90	±60	±30	0.9	**0.688**	**+0.646**	0.690	−0.017

Neither ingredient works alone. A **full hue circle at the default probability** reaches only 0.261, and **half the hue range with the full probability and saturation** reaches 0.177 — while both together reach 0.688. The gain is roughly 3× what either part contributes, so this is an interaction and not a dose-response curve: the model needs both a hue range wide enough to reach amber *and* a probability high enough that it cannot fall back on the unshifted half of the data.

Per-seed Bachsee: mid45 0.203/0.113/0.216, hueonly 0.182/0.428/0.172 (note the spread — 0.428 on one seed, so hueonly is unstable as well as insufficient).

The in-domain cost scales with the same combination: −0.001 (hueonly), −0.002 (mid45), −0.017 (strong). Buying the +0.646 costs 1.7 F1; the cheaper settings save nothing worth having because they do not deliver the gain.

Recommendation: `--aug-hue-shift 90 --aug-sat-shift 60 --aug-val-shift 30 --aug-hsv-p 0.9`. Do not soften it.

This retires the standing advice that a site holdout is impossible on this corpus. Untested: whether this transfers to spruce and pine, and whether p alone (hue ±20 at p 0.9) does anything — the two arms here vary p together with sat. Failing that, this site should never be the held-out one.

5.2.2 Kaufland is the informative clean holdout

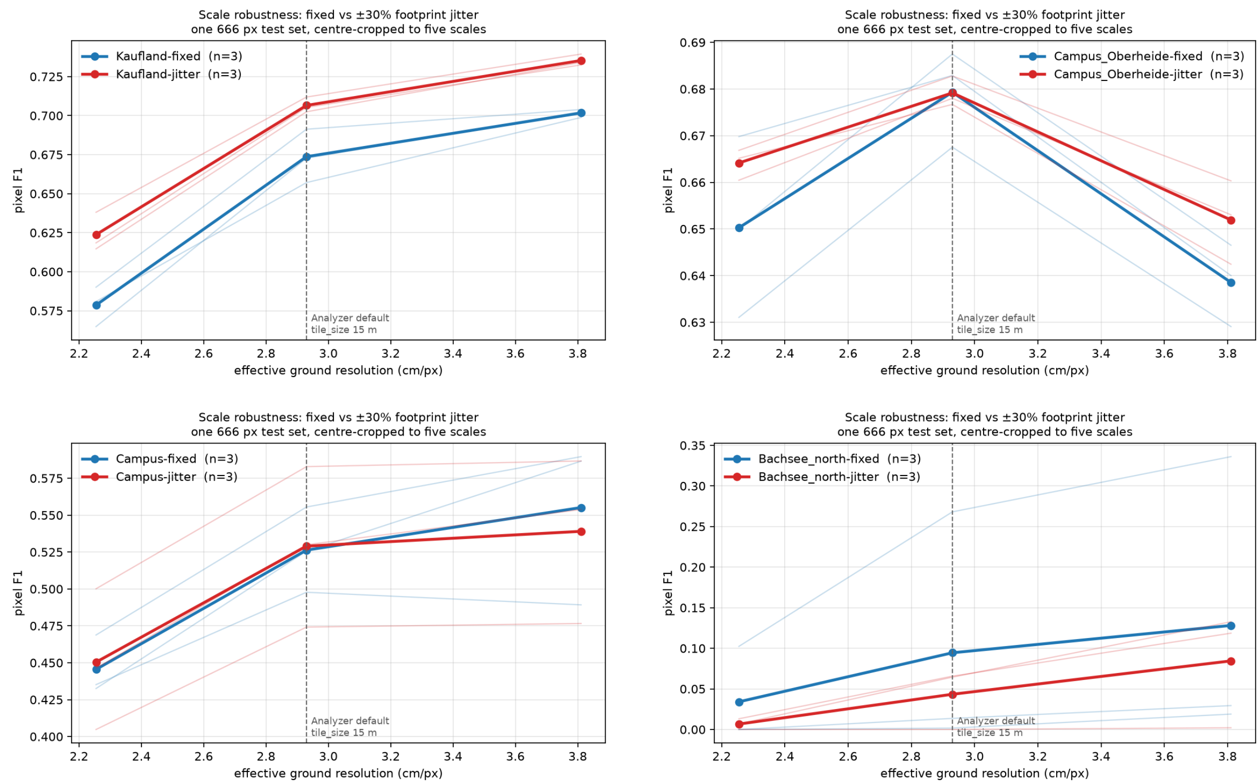
The only true spatial holdout where the model works at all, and jitter wins **at every scale on every seed**:

ratio	GSD	fixed	jitter	diff	sign	t
0.77×	2.254	0.5786	0.6237	**+0.0451**	3/3	4.37
1.00×	2.930	0.6737	0.7065	**+0.0328**	3/3	2.61
1.30×	3.811	0.7017	0.7353	**+0.0336**	3/3	9.58

Here jitter is worth **+1.1 / +3.3 / +5.5 F1 across the three seeds** (mean +3.3, 3/3 positive, |t| = 2.6 — below the |t| >= 3 bar used elsewhere here, so the direction is reliable and the magnitude is not) at the deployment scale — an order of magnitude more than the +0.2 measured within-site, though on a single plot. Note the curve *rises* with coarseness on this fold, unlike every other: Kaufland is the corpus's finest orthomosaic (2.09 cm/px native) and its densest (stem fraction 0.063 against 0.028–0.038), so its peak sits outside the sampled range. Its spread is therefore a lower bound.

5.3 The curves

Per fold, F1 against effective ground resolution, per-seed spread drawn rather than summarised. Note the y-scales differ by an order of magnitude between folds.



5.4 Does Bachsee_north belong in training? Yes — tested

Before it was understood as domain shift, Bachsee looked like 800 tiles of unlearnable noise sitting in ~20–29% of every training set. So the folds were re-run with Bachsee **excluded from training entirely** (3 folds x 2 arms x 3 seeds = 18 UNet runs, everything else identical). Paired within fold, arm and seed, at 1.00x:

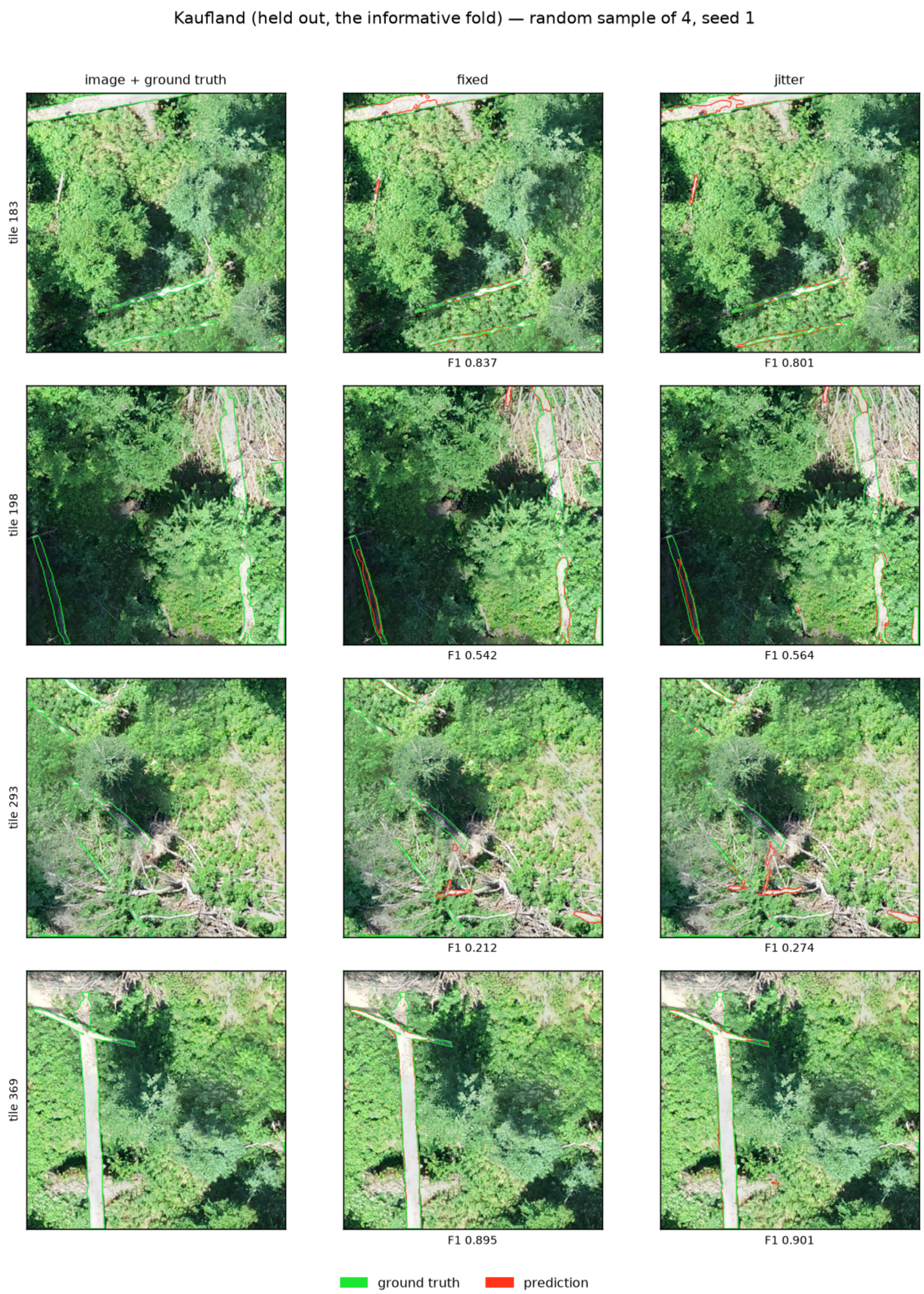


Figure 8: Kaufland examples

fold (acquisition)	train tiles	F1 with Bachsee	without	delta	signs
Kaufland (Jul, summer green)	4000 -> 3200	0.6901	0.6953	**+0.005**	5/6
Campus_Oberheide (Feb, leaf-off)	2788 -> 1988	0.6793	0.6338	**-0.045**	0/6
Campus (Oct, autumn)	2788 -> 1988	0.5277	0.4324	**-0.095**	0/6

Overall **-0.045 F1**, 5 of 18 paired comparisons positive. **Removing Bachsee hurts.**

Declared confound: dropping the site also drops 800 training tiles, so Campus and Campus_Oberheide lose 29% of their data and Kaufland 20%. Quantity alone would predict the direction. It does not predict the *ordering*: Campus and Campus_Oberheide lose the **identical** 800 tiles from identical 2788-tile sets, yet Campus loses twice as much F1. What separates them is phenological distance from Bachsee's November amber — Campus was flown in October, Campus_Oberheide in leaf-off February, Kaufland in high summer, and the damage falls in exactly that order.

So the out-of-domain site is not noise to be removed; it is the nearest thing the corpus has to phenological diversity, and the sites that resemble it depend on it most. In a corpus with under 1 ha of labelled stem, discarding an acquisition is expensive.

The arm comparison is unaffected in direction: jitter's curve stays flatter in 3/3 folds without Bachsee (-0.0142, -0.0450, -0.0012), though at 1.00x it now loses on the two Campus folds and wins only on Kaufland — consistent with those folds simply having less data to work with.

5.5 What the holdout costs

	within-site (pooled blocks)	site holdout
F1 @ 1.00×	0.78	0.04 – 0.71

Between 7 and 74 F1 points, depending entirely on which site. Any single-fold number here would have been arbitrary — which is the argument for reporting all four.

5.6 Caveats

- **n = 3 seeds per arm per fold.** Under domain shift the within-arm seed spread reaches **12 F1 points** (Campus fold), against 1–2 within-site. Absolute F1 differences below ~3 points are unreadable at this n; the spread statistic is within-model and survives better, which is why the claim rests on it.
- **Only 2 of 4 folds are true spatial holdouts**, and one of those is dead. The new-site evidence is effectively **one fold** — strong, but one.
- **Train sizes differ by fold** (2788–4000 tiles), inherent to leave-one-site-out.
- **Val comes from one site** in the Campus and Campus_Oberheide folds, since only those two sites are large enough to block-split at a 19.512 m footprint.
- **Corpus limit.** Four beech sites, two of them the same ground. A stronger claim needs annotated sites this corpus does not have — see winmol-label-scarcity.

5.7 Verdict

The within-site recommendation stands and is mildly strengthened: jitter never hurt in any fold, flattened the curve in all four, and on the one clean holdout where segmentation works at all it gained accuracy (per-seed +1.1/+3.3/+5.5 F1; direction consistent, magnitude poorly determined). But calling this "validated across sites" would overstate a corpus with one informative holdout fold.

6 Revier 12 / 13 (Tegel) — zero-shot, fine-tune and native-resolution results

Design and the frozen test set were fixed before any model ran — pre-registered in `specs/2026-08-16-tegel-r12-r13-design.md`, which now lives in the private helper repo. All numbers below are on that same held-out ground.

Headline: existing beech models already reach F1 0.76 on this new survey with no Tegel data at all. Training on Tegel adds +3.5 to +6.2 F1. Fine-tuning from the beech model is worth nothing over training from scratch. And **native-resolution extraction loses to 2.93 cm on every seed and both plots** through the deployment path — settled below.

6.1 The data

Two July-2025 surveys, digitised as five widely separated sample plots.

plot	polygons	stem area	AOI	role
R12-P1	178	215 m²	14,913 m²	val
R12-P2	552	891 m²	23,080 m²	train
R12-P3	299	501 m²	8,692 m²	**test (frozen)**
R13-P1	232	248 m²	8,865 m²	train
R13-P2	123	141 m²	13,436 m²	**test (frozen)**
total	**1,384**	**1,996 m² = 0.20 ha**		

That is a **20% increase on the entire pre-existing corpus** (0.98 ha), and it is far more mixed than the beech sites:

species	polygons	species	polygons
beech	897	hornbeam	12
pine	**153**	poplar (3 spellings)	16
(no species)	121	spruce	7
birch	120	silver fir	1
ash	41	oak	16

153 pine polygons, in a corpus that previously had no annotated pine stand. Note `POPLAR` / `WHITE POPLAR` / `SILVER POPLAR` are three spellings of one group — the same naming-variant trap the older corpus has with `RBU`/`rBU`/`RBu`.

Orthomosaics: R12 1.20 cm/px (339k × 334k px, 12 GB, RGBA), R13 1.28 cm/px (393k × 335k px, 19 GB). Both finer than anything else held — Kaufland's 2.09 cm/px was previously the finest.

6.2 Registration verified first

Labels are `EPSG:25833`, orthomosaics `EPSG:32633` — the exact datum mismatch that made Bachsee_north look like a dead site. Checked before anything was trained or scored, on each orthomosaic's own unrotated grid:

plot	contrast at zero shift	bright peak	verdict
R12-P2	+1.205	(0.00, 0.00) m	registered
R12-P3	+1.094	(0.00, 0.00) m	registered
R13-P1	+0.894	(0.00, 0.00) m	registered
R13-P2	**+1.748**	(0.00, 0.00) m	registered

All zero-offset, and **stem contrast is higher than anywhere in the beech corpus** (Kaufland, the best, was +0.89). This is high-quality data.

6.3 Results — per plot, never pooled

Test tiles cut at 19.512 m / 666 px (2.9297 cm/px), scored at the 1.00× centre window. Three seeds per trained arm; the value is the mean.

arm	R12-P3	R13-P2	scored at
zero-shot — beech UNet (jitter)	0.7636	0.6081	2.93 cm
zero-shot — beech HRNet (jitter)	0.7624	0.6313	2.93 cm
Tegel from scratch	**0.7988**	0.6696	2.93 cm
Tegel fine-tuned from beech	**0.8031**	0.6719	2.93 cm
Tegel native	0.7861	**0.6798**	*1.20 / 1.28 cm — different exam*

Per seed (R12-P3 / R13-P2): scratch 0.7987·0.7970·0.8008 / 0.6659·0.6704·0.6725; fine-tune 0.8001·0.7965·0.8128 / 0.6676·0.6604·0.6877; native 0.7928·0.7688·0.7967 / 0.6909·0.6559·0.6925.

6.3.1 Zero-shot already works

0.76 on R12-P3 without a single Tegel tile in training, against 0.78–0.79 for the beech models on their own within-site test data. A genuinely new survey, a different season, a mixed stand with pine and birch, and imagery 2.4× finer than the training scale — and the model transfers. That is what matching the training GSD to the serving GSD buys, and it is the strongest evidence yet for that rule.

R13-P2 is harder (0.61–0.63 zero-shot). It is the most mixed plot: 51% birch, 20% beech, and 15% with no species recorded.

6.3.2 Training on Tegel is worth +3.5 to +6.2 F1

Scratch minus zero-shot: **+3.5** on R12-P3, **+6.2** on R13-P2. The larger gain on the harder, birch-dominated plot is the expected shape — the corpus had no birch stand.

6.3.3 Fine-tuning buys nothing over training from scratch

+0.4 F1 (R12-P3) and +0.2 F1 (R13-P2), both inside the per-seed spread. With 2,330 Tegel tiles, the beech initialisation is not doing measurable work. This is a useful negative: for a survey of this size, a clean run is as good as a transfer, and simpler. `--init-weights` was added for this arm and loads all 31,036,673 parameters or refuses.

6.3.4 Native resolution — SETTLED: it loses, on both plots, on every seed

Resolved by running both models through the **WINMOL Analyzer**, each at its own matched `tile_size` (15 m for the 2.93 cm models; 6.14 m on R12 and 6.55 m on R13 for the native models, since 512 source px is 6.14 m at 1.20 cm/px). Both predictions were scored identically: resampled to one 2.93 cm reference grid, masked to the plot AOI shrunk by 2 m, via `scripts/score_stem_map.py`. Three seeds each.

plot	2.93 cm	native	delta	seeds favouring native
R12-P3	**0.7832**	0.7653	**-0.0178**	0/3
R13-P2	**0.5840**	0.5482	**-0.0358**	0/3

Per seed — R12-P3: 0.7832/0.7786/0.7877 against 0.7710/0.7464/0.7786. R13-P2: 0.5742/0.5952/0.5826 against 0.5506/0.5343/0.5598. **Native loses all six pairings.**

The mechanism is precision, and it is consistent. Native has *higher* recall on all six runs and *lower* precision on all six:

	recall	precision
R12-P3	native 0.79–0.85 vs 0.79–0.85	native **0.71–0.73** vs 0.73–0.77
R13-P2	native **0.70–0.74** vs 0.67–0.68	native **0.43–0.45** vs 0.50–0.54

More pixels per stem finds more stem — and more things that merely look like stem. The net is negative at both plots.

It is also far more expensive: at 6.14 m tiles the Analyzer processed **676 tiles per plot against 11**, roughly 60× the inference for a worse result.

Recommendation: extract at the serving scale (2.93 cm/px), not native, even where the imagery is 2.4× finer and even though the Analyzer's `tile_size` can be changed to match.

6.3.5 New model vs the published WINMOL model, through the Analyzer

All four models run through the Analyzer at `tile_size 15` and scored identically — resampled to one 2.93 cm reference grid, masked to the plot AOI shrunk by 2 m.

model	R12-P3	R13-P2
published — Reder `SpecDS_Beech_512` (2023-02-28)	0.7435	0.5715
ours, beech corpus — BeechScale666 UNet, no Tegel data	0.7404	0.5252
ours, Tegel from scratch (3-seed mean)	**0.7832**	0.5840
ours, Tegel fine-tuned	0.7811	**0.5930**

On these two plots the published model looks better. It matches ours on R12-P3 (0.7435 vs 0.7404) and beats it by **4.6 F1** on R13-P2 (0.5715 vs 0.5252). *This reading does not survive the other three plots* — see the five-plot comparison below, where the two models come out indistinguishable (mean difference –0.008) and R13-P2 turns out to be the one plot whose labels are too incomplete to score against. Treat the two-plot gap as a sampling artefact, not a transfer result.

Two things this does not mean. It is not a like-for-like architecture comparison — the published model is a different training corpus (including sites we do not hold, such as Quesenbank) as well as a different pipeline. And it is a single plot pair; the two plots disagree in magnitude.

What does help is local data. Training on Tegel beats the published model by **+4.0 F1** on R12-P3 and **+2.2** on R13-P2. The gain comes from labelling the survey you intend to run on, not from a better-scaled general corpus.

6.3.6 The beech-corpus model on all five plots, from the full orthomosaics

The table above compares models on the two frozen test plots. This extends it to **all five** digitised plots, scoring the full-orthomosaic stem maps rather than plot windows. Numbers copied verbatim from `assets/beech-corpus-zeroshot.json`; same basis throughout (Analyzer at `tile_size 15`, one 2.93 cm reference grid, AOI shrunk by 2 m).

plot	beech share	ours, beech corpus	published	ours, Tegel-trained
R12-P1	93%	0.6718	0.6796	0.7188
R12-P2	81%	0.7304	0.7298	0.7689 *
R12-P3	72%	0.7408	0.7449	0.7841

plot	beech share	ours, beech corpus	published	ours, Tegel-trained
R13-P1	20%	0.6722	0.6529	0.7419 *
R13-P2	20%	0.5218	0.5686	0.5694

* trained on that plot — not a held-out score, and not comparable to the other two columns.

Our beech-corpus model and the published one are indistinguishable. Per-plot differences are −0.008, +0.001, −0.004, +0.019, −0.047: signs split two to three, mean −0.008. That is well inside the 0.45–0.96 noise floor this repo measures with `--deterministic`, so the earlier "the published model out-transfers ours" reading — drawn from R13-P2 alone — does not survive the other four plots. The honest statement is that two independently trained beech models reach the same accuracy on new ground.

This also re-validates the harness. The two plots that appear in both tables agree across completely different inference paths — plot windows versus the full orthomosaic: R12-P3 0.7408 against 0.7404, R13-P2 0.5218 against 0.5252.

Both zero-shot models fail by over-predicting, which F1 hides. Precision and recall for the beech-corpus model:

plot	precision	recall
R12-P1	0.5728	0.8122
R12-P2	0.7180	0.7431
R12-P3	0.7131	0.7707
R13-P1	0.7829	0.5890
R13-P2	0.4574	0.6073

Recall exceeds precision on all three R12 plots. That is the benign failure mode — a human filtering false positives has an easier job than one hunting missed stems — and it is the opposite of the recall collapse seen under colour-domain shift.

R13-P2 should not be read as a model score. It is the plot where 43% of predictions are unmatched because the labels are incomplete, so both zero-shot models are penalised for finding real stems that were never digitised. Excluding it, and excluding the two plots the Tegel model trained on, **local training is worth about +4 F1** (R12-P1 +4.7, R12-P3 +4.3).

6.3.7 Cross-validation of the harness

The world-space evaluator built for this comparison (`scripts/plot_inference.py`) and the Analyzer are independent implementations — different tiling, different stitching, different codebase. On the same model and plot they agree to **0.001 F1** (0.7842 vs 0.7832), which is why the numbers above can be trusted.

6.3.8 Tile-level F1 flatters — by a lot on the sparse plot

The same models score very differently depending on whether you measure on sampled tiles or over the whole AOI:

	tile-level	world-space, whole AOI
2.93 cm @ R12-P3	0.7988	0.7832
2.93 cm @ R13-P2	0.6696	**0.5840**

Training tiles are sampled with a minimum stem fraction, so tile-level scoring over-represents stem-rich ground and never asks the model about the empty areas where its false positives live. On R13-P2 that is worth **8.6 F1** of optimism. Deployment numbers should come from AOI-masked world-space scoring.

6.3.9 The superseded framing

The tile-level table above scores the native arm on its own 1.20/1.28 cm tiles, where a stem is ~2.4× thicker in pixels — a different exam, and not a valid comparison. On that easier exam native appeared to win R13-P2 and lose R12-P3. The world-space result above supersedes it: on a common grid native loses both.

6.4 Reproducibility: the whole pipeline rebuilt and retrained identically

After the NAS mount on T14 dropped and was restored, both datasets were re-extracted and all six models retrained from scratch. Every run reproduced its original to four decimal places:

seed	2.93 cm rerun	2.93 cm original	native rerun	native original
s1	0.7727	0.7727	0.7651	0.7651
s2	0.7726	0.7726	0.7393	0.7393
s3	0.7770	0.7770	0.7688	0.7688

Precision and recall match as well, 6/6.

This is a stronger check than a rerun of the same command, because several things changed between the two passes and all had to be right at once:

- **A different storage path** — the NAS was remounted with `cifs-utils` freshly installed and different mount options (SMB 3.1.1). A resampling or byte-level difference in how the imagery was read would have moved the numbers.
- **A different extraction code path** — the rerun used `scripts/extract_parallel.py`, five concurrent processes merged with renumbering, against the original's single-process `make_splits`. A tile-index collision or a dropped plot in the merge would have shown up here.
- **Different hosts** for tiling and training than the first pass.

Before training, the tile sets were also compared directly: identical centres and rotations in every split, 2330/500/1000 and 2400/500/1000.

So the parallel extractor is verified against the serial one not only on tile counts and coordinates but on the models those tiles produce — and the ranking below holds on freshly extracted data.

6.5 Full-orthomosaic inference — both models, both Reviere

The plot-level numbers above are cut from 155–181 m windows. This is the whole deliverable: every tile of both orthomosaics, through the Analyzer, for our Tegel model and the published one.

	R12	R13
rectangular extent	16.3 km²	21.6 km²
actually imaged (footprint of valid pixels)	**5.15 km²**	**10.33 km²**
fill of the rectangle	**33.8%**	**51.1%**
tiles at `tile_size 15`	75,072	99,231
new model — stems	**8,323**	**25,238**
published model — stems	6,483	20,141
difference	**+28%**	**+25%**
new-model nodes (a diameter every 50 cm)	75,256	232,383

Quote densities against the imaged area, not the rectangle. These orthomosaics are flight strips, not filled rectangles — R12 covers only a third of its bounding box. Using the rectangle understates stem density threefold:

	stems/km² imaged	stems/km² if you use the bbox
R12, new model	**1,616**	546
R13, new model	**2,442**	1,248

The exact outlines are `R{12,13}_footprint.gpkg` (one polygon, ~59k vertices, no holes), produced with `gdal_footprint -ovr 3`, plus a full-resolution validity raster `R12_valid_mask.tif` from the alpha band. It also means roughly **two-thirds of the R12 inference was spent on empty pixels** — masking to the footprint first would cut the full-orthomosaic run substantially.

More stems is not itself evidence of a better model. Outside the five sample plots nothing was digitised, so those extra detections cannot be judged — scoring the whole raster would count correct detections as false positives. The defensible comparison stays the AOI-masked one.

6.5.1 The full run reproduces the plot run

Restricting the full-orthomosaic output to the frozen test plots recovers the earlier numbers, which validates the whole path — different host, different tiling grid, and a different preprocessing route:

model @ plot	full-ortho	earlier crop run	delta
new @ R12-P3	0.7841	0.7832	+0.0009
published @ R12-P3	0.7449	0.7435	+0.0014
new @ R13-P2	0.5694	0.5742	−0.0048
published @ R13-P2	0.5686	0.5715	−0.0029

All four within 0.5 F1, which is what tiling-offset and edge effects alone would produce.

It also settles a question raised by the deployment path: the full runs used an **in-graph preprocessing wrapper** (normalise + bicubic resize prepended to the ONNX graph) while the crop runs did the resize on the CPU. Against a direct reference the wrapper agreed at **IoU 0.9964** with 0.05% of pixels flipping the 0.5 decision, and the table above confirms that at the level of the final score.

One nuance the counts hide: the new model's advantage is plot-dependent. On R12-P3 it leads by ~3.9 F1; on R13-P2 the two are **tied** (0.5694 vs 0.5686). R13-P2 is the birch-dominated plot with 15% of polygons carrying no species.

6.5.2 Throughput, and where to run this

host / mode	tiles/min	R12 wall time
Mac (CoreML)	57	~22 h
carrot, CPU (224 cores)	388	~3 h
T14 (RTX 4080 SUPER)	1,206	~1 h
carrot (H100, ORT-GPU)	**4,576**	**27 min**

carrot needed `onnxruntime-gpu` installed — its environment had the CPU build, which is why the first attempt there was 12× slower than a laptop GPU despite 224 cores. This workload is GPU-bound, not core-bound.

Outputs: `R{12,13}_{new,old}.gpkg` with `stems`, `nodes` and `vectors` layers, plus stem-map rasters. A QGIS project comparing them is generated by `make_qgis_project.py` (private helper repo).

6.6 Deviation from the pre-registered spec

The spec said the fine-tune would use strong hue augmentation (`--aug-hue-shift 90 … --aug-hsv-p 0.9`). **It did not** — all three trained arms ran with the default colour augmentation. The reason is that train and test here are the same survey and the same season, where the strong setting is measured to *cost* 1.7 F1; its +64.6 F1 benefit is out-of-domain. Applying it to only some arms would also have confounded the native comparison.

Stated because the spec was written first and this differs from it. Untested consequence: these Tegel models will likely transfer worse to a different season than a strong-augmentation version would.

6.7 Caveats

- **Two test plots.** 422 polygons. Reported separately throughout; the two disagree about the native arm, which is exactly why they are not pooled.
- **Mixed species.** These stands are 65% beech, 11% pine, 9% birch. A zero-shot gap may be species rather than site.
- **Season.** July, against a beech corpus that is mostly autumn and winter. Kaufland (July) is in the training corpus, which likely helps.
- **Colour augmentation left at defaults** for all arms, since train and test are the same season here. The strong setting costs 1.7 F1 in-domain and its benefit is out-of-domain.
- **Train/test share Reviere.** R12-P2 trains and R12-P3 tests; plot centroids are ~600 m apart. Leak-free (no shared ground) but not a new-forest claim.

7 Working process

7.1 Splits

Standard: no tile footprint may overlap between train, val and test. Spatial autocorrelation inside a split is accepted and expected.

`make_splits.py --strategy blocks` satisfies this by construction: the AOI is cut into a grid, whole blocks are dealt to splits, and each block is then shrunk by the footprint half-diagonal. Two centres on opposite sides of a shared edge are therefore at least `extent · √2` apart — the distance below which two rotated footprints can share a pixel.

Verification is a pass/fail check, not a discussion. `plot_splits.py` (private helper repo) prints the measured closest centre-to-centre distance per site and exits non-zero if a site is missing from a split. Run it after every build; report the number once.

	requirement
footprint overlap between splits	none — closest centre distance > `extent · √2`
spatial autocorrelation within a split	accepted
every site in every split	yes, unless the AOI is too small to cut (record in `splits.json`)
provenance	`tiles.jsonl` per split: site, centre, angle, GSD, stem fraction

A site whose AOI cannot yield 12+ usable blocks goes wholesale into one split via `whole_split`, recorded in `splits.json`. No further comment needed.

7.2 Scale

Train at the scale the Analyzer serves. Effective GSD is `extent_m / tile_px` in training and `tile_size / 512` in the Analyzer. These must match, or the model is served at a different resolution than it learned.

Default `tile_size = 15` -> **2.93 cm/px** -> build with `--extent 15 --tile-px 512`.

Measured on Kaufland, AOI-masked, same model family:

trained at	served at	F1
2.00 cm/px	2.93 cm/px (mismatched)	0.7389
2.00 cm/px	2.00 cm/px (matched)	0.8091
2.93 cm/px	2.93 cm/px (matched, default)	**0.8844**

Ship the `tile_size` alongside any model that is not trained at 2.93 cm/px.

Jitter the training footprint ±30% — `--multiscale --crop-min-px 394 --crop-max-px 666` on a 666 px source. Measured on two architectures, 3 paired seeds each: it flattens the F1-vs-GSD curve by 0.87 F1 on HRNet and 1.88 on UNet, and costs nothing at the native scale. See `scale-augmentation-results.md`. Matching the scale is still worth far more than being robust to it — do both.

7.3 Evaluation

- Tile-level F1 for comparing training runs.
- **Full-orthomosaic inference masked to the AOI** for anything claiming deployment performance. Outside the windthrow polygon stems are real but undigitised; scoring the whole raster is meaningless (measured: 0.2974 vs 0.8844 on the same prediction).
- Report precision and recall separately.

7.4 Reporting

- Numbers come from each run's own `test_results.md`, collected by `collect_results.py` (private helper repo) into JSON. Reports render from that JSON.
- State the test set and the training scale beside every figure.
- One line per caveat, in a caveats section. Not inline.

7.5 Pipeline order

`fix -> sample -> split`, enforced by `make_splits.py`. Geometry repair happens first because GEOS aborts on the first set operation touching an invalid ring.

7.6 Environments

what	where
corpus	`/Volumes/storage/Datasets/Winmol/training_data/WINMOL_Trainings_Da`
derived tiles	`data/` in this repo, git-ignored, regenerable from `configs/*.json`
training	the GPU box (`$WINMOL_CARROT`) — check GPUs are free before launching, and pin them with `--gpus '"device=4,5,6"'` rather than taking the whole machine
Analyzer	`~/hnee/WINMOL_Analyzer`, needs Python ≥ 3.10